# Problem1 - Verilog and Logic Design

## Background

Multiplication-Accumulation (MAC) module is the central computing element for the emerging on-chip neural network accelerator. In this lab, you will design a MAC module with pipelined operation.

# Description

a. Inputs: IN (signed 4 bits), W (signed 4 bits, weights for neural network), clk (clock), rstb (resetb, reset at 0);

b. Output: OUT (signed 12 bits);

c. At every clock cycle, your MAC will perform IN*WEIGHT and accumulate with its internal stored value from previous cycle to generate the OUT value.

d. The accumulator should start from 0 value (for example, A and B are set to 0) at the beginning and perform accumulation for 9 cycles (the reason for 9 cycles is that we usually have 3x3 image filter as weight for deep neural network); After that, B starts from 0 again while A takes the new input from Multiplier. Essentially, the accumulator accumulates every 9 clock cycles and restart the accumulation from the next cycle. The OUT will generate final MAC output results every 9 clock cycles (after a few clock cycles' delay). You need to decide how many bits A and B signals should have to accommodate the value range of the total.

e. The MAC has a pipelined structure as Figure 1 below (FSM or counter is not shown).

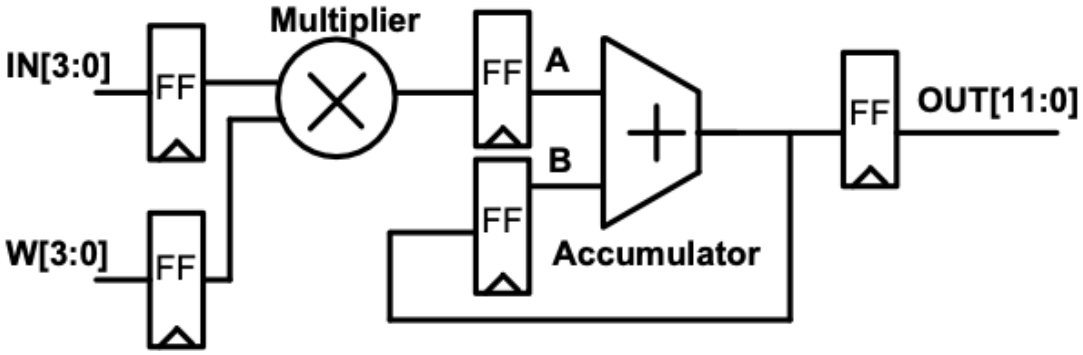


Figure 1: problem 1 pipelined structure

f. The MAC design runs at 1GHz clock;

g. All inputs (except clk, resetb) need to satisfy a max input delay of 0.5ns and a minimum input delay of -0.2ns from external modules (The negative minimum delay is used to add additional hold margin for the input signal paths);

h. All output needs to satisfy a max output delay of 0.5ns and a minimum output delay of -0.2ns (The negative minimum delay is used to add additional hold margin for the output signal paths);

i. Upon reset (resetb =0), all internal flip-flops are reset to 0;

# Deliverable

## 1

Following the lectures, create all required design files including RTL of MAC, testbench, timing constraint file, and other needed files.

## 2

Use Xcelium to simulate your RTL using your testbench. Show the snapshots of the simulated waveforms (show all inputs and outputs signals) for two rounds of operations, e.g. 20 clock cycles. The inputs should change every clock cycle (It is your choices of what IN and W to give, but cannot be all zeros). Show the outputs are calculated correctly.

**Answer:**

These two figures shown in Figure **??** the functional correctness of the pipelined MAC design. The waveform captures verify that each accumulation window consists of 9 valid multiply operations, and the final accumulated result is produced at the expected cycle.

a. Cycle 1

$$1 - 18 + 6 + 12 - 6 - 21 + 8 - 36 - 36 = -90$$

b. Cycle 2

$$42 - 3 - 6 - 35 + 9 - 10 - 24 + 15 = -12$$

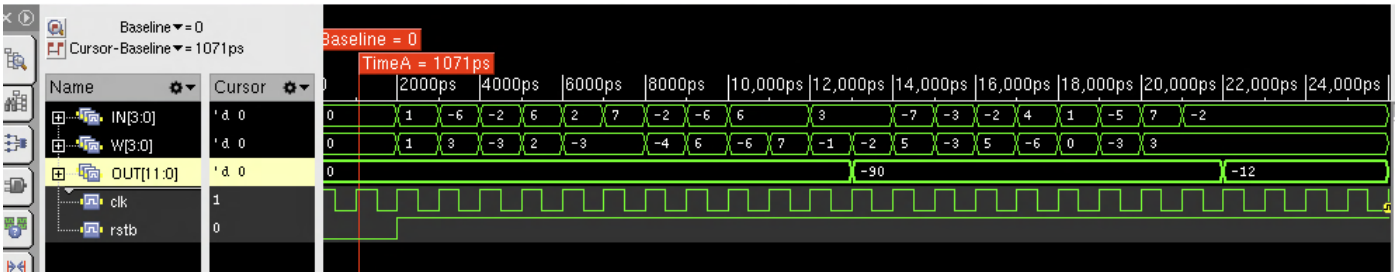


Figure 2: Waveforms of the RTL MAC design over two accumulation windows.

## 3

Perform synthesis following Genus tutorial. Report your area, cell counts, and timing slack. Attach snapshot of *report_area* and *report_timing* output.

**Answer:**

- Cell count: 167
- Total area: $\boxed{625.964}$ (library units)
- Worst-case path delay: 930 ps
- Required time: 967 ps

- Slack: 37 ps

~/CE303/HW5/Problem1/bxrun_rtl.sh

```
  ============================================================
  Generated by:           Genus(TM) Synthesis Solution 18.14-s037_1
  Generated on:           Nov 18 2025  08:40:50 pm
  Module:                 mac
  Operating conditions:   typical
  Interconnect mode:      global
  Area mode:              physical library
  ============================================================

Path 1: MET (37 ps) Setup Check with Pin acc_reg_reg[11]/CK->D
           Group: clk
      Startpoint: (R) in_reg_reg[3]/CK
           Clock: (R) clk
        Endpoint: (R) acc_reg_reg[11]/D
           Clock: (R) clk

                    Capture       Launch
         Clock Edge:+    1000            0
        Src Latency:+       0            0
        Net Latency:+       0 (I)        0 (I)
            Arrival:=    1000            0

              Setup:-      33
      Required Time:=     967
       Launch Clock:-       0
          Data Path:-     930
              Slack:=      37

#-------------------------------------------------------------------------------------------------------
#   Timing Point    Flags    Arc   Edge   Cell      Fanout Load Trans Delay Arrival Instance
#                                                        (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------
  in_reg_reg[3]/CK  -        -      R     (arrival)     37    -     0     -       0    (-,-)
  in_reg_reg[3]/QN  -        CK->QN F     DFFR_X1        4  6.4    17    76      76    (-,-)
  g3602__1474/ZN    -        A2->ZN F     OR2_X1         1  3.6    11    58     133    (-,-)
  g3552__9906/S     -        B->S   F     HA_X1          1  3.0    13    59     192    (-,-)
  g3531__2683/S     -        CI->S  R     FA_X1          1  4.0    16   117     309    (-,-)
  g3513__2703/S     -        A->S   F     FA_X1          3  5.2    19    95     404    (-,-)
  g3504__5266/CO    -        A->CO  F     FA_X1          1  3.0    15    80     484    (-,-)
  g3488__7675/CO    -        CI->CO F     FA_X1          1  3.0    15    72     556    (-,-)
  g3482__5019/CO    -        CI->CO F     FA_X1          1  3.0    15    72     628    (-,-)
  g3478__2703/CO    -        CI->CO F     FA_X1          1  3.0    15    72     701    (-,-)
  g3474__5266/CO    -        CI->CO F     FA_X1          3  5.8    19    79     780    (-,-)
  g3472__7114/ZN    -        A1->ZN R     NAND3_X1       2  3.7    16    26     805    (-,-)
  g3466__7118/ZN    -        C1->ZN F     OAI211_X1      1  2.4    17    23     828    (-,-)
  g3460__1309/ZN    -        A->ZN  F     XNOR2_X1       1  1.8    12    41     870    (-,-)
  g3455__1474/ZN    -        B1->ZN R     OAI21_X1       2  1.4    21    26     896    (-,-)
  g3451__8780/ZN    -        A1->ZN R     AND2_X1        1  1.4     9    34     930    (-,-)
  acc_reg_reg[11]/D <<<     -      R     DFFR_X1        1    -     -     0     930    (-,-)
#-------------------------------------------------------------------------------------------------------
```

Figure 3: Post-synthesis timing report of the MAC. The critical path from `in_reg_reg[3]` to `acc_reg_reg[11]` has a data path delay of 930 ps under a 1 GHz clock, resulting in a positive setup slack of 37 ps.

```
Problem1 > area.rpt
1  ============================================================
2    Generated by:           Genus(TM) Synthesis Solution 18.14-s037_1
3    Generated on:           Nov 18 2025  08:41:08 pm
4    Module:                 mac
5    Operating conditions:   typical
6    Interconnect mode:      global
7    Area mode:              physical library
8  ============================================================
9
10 Instance Module  Cell Count  Cell Area  Net Area   Total Area
11 ------------------------------------------------------------
12 mac                    167    395.276   230.688      625.964
13
```

Figure 4: Post-synthesis area report of the MAC. The synthesized design contains 167 standard cells with a total cell area of 625.964 library units.

## 4

Repeat the step (2) by simulating your generated gate level netlist. Correct functions should be shown.

**Answer:**

After completing logic synthesis with Genus, the generated gate-level netlist was simulated using Xcelium to verify post-synthesis functionality in Figure 5. The gate-level waveform confirms that the MAC unit still produces a correct accumulated output every 9 clock cycles, matching the behavior of the RTL model. All input transitions and accumulation results propagate correctly through the synthesized logic.

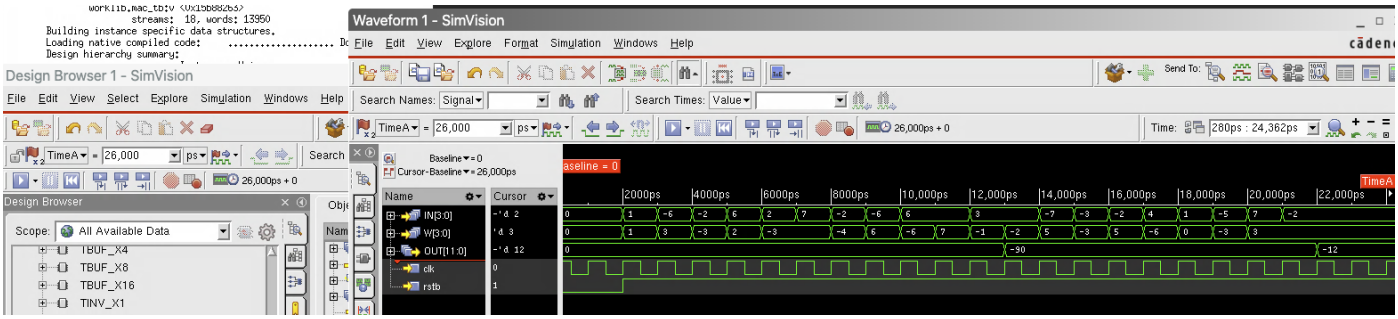


Figure 5: Gate-level simulation results of the synthesized MAC, showing correct functional behavior matching the RTL model.

## 5

Please attach all supporting files in your submission including RTL, testbench, SDC file. They can be pasted into the same file of your report or be attached to your submission as separate files.

**Answer:**

I will attach the testbench and the rtl verilog file in submission, while the sdc will be shown in Figure 6

Problem1 > mac.sdc

```
# 1 GHz (1 ns period)
create_clock -name clk -period 1.0 [get_ports clk]

# Input delays for IN and W
set_input_delay -max 0.5 -clock clk [get_ports {IN[*] W[*]}]
set_input_delay -min -0.2 -clock clk [get_ports {IN[*] W[*]}]

# Output delays for OUT
set_output_delay -max 0.5 -clock clk [get_ports {OUT[*]}]
set_output_delay -min -0.2 -clock clk [get_ports {OUT[*]}]

```

Figure 6: mac adc file

### 6

Hint: For setting input delay and output delay of the ports, an example of commands to use in your SDC file is below. You may need to replace the port and clock names with the names in your design.

```
set_input_delay -max 0.5 -clock clk [get_ports {IN W}]
set_input_delay -min -0.2 -clock clk [get_ports {IN W}]
set_output_delay -max 0.5 -clock clk [get_ports {OUT}]
set_output_delay -min -0.2 -clock clk [get_ports {OUT}]
```

## Problem 2 - Sequential Circuits

Use the circuit diagram in Figure 7 below with the following parameters:

1. $t_{pcq} = 10$ ps;
2. 5 ps$< t_{p,AND} <$ 10 ps;
3. 5 ps$< t_{p,OR} <$ 10 ps;
4. 10 ps$< t_{p,XOR} <$ 20 ps;
5. 2 ps$< t_{p,INV} <$ 5 ps;

Both flip-flops are identical rising-edge DFFs. Assume the values of x and y are always stable from one rising clock edge to the next rising clock edge.

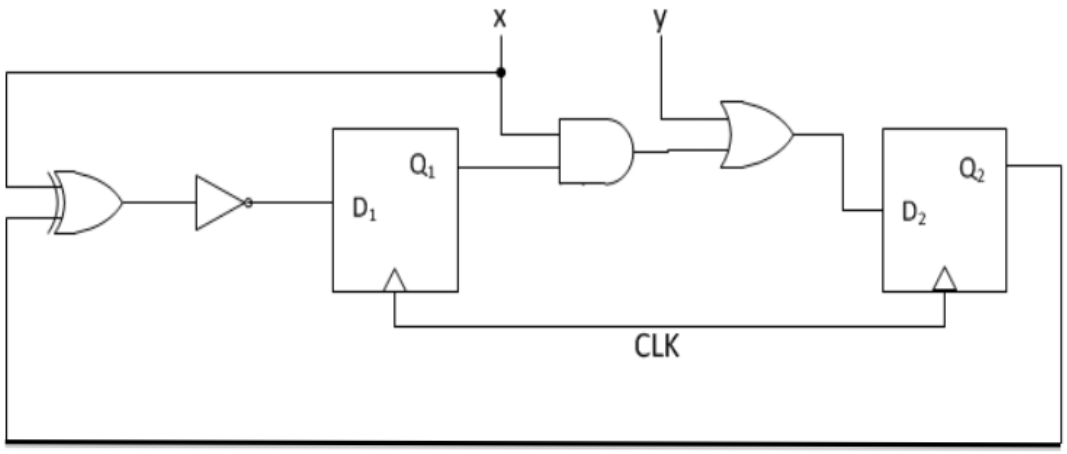


Figure 7: problem 2 schematic

## a

What is the maximum setup time of the flip-flops in order to safely use a 50 ps clock?

**Answer:**

The setup constraint defines the relationship between clock period and the worst case delay of combinational logic, which is define by

$$T_{clk} > t_{pcq} + t_{setup} + t_{pdmax}$$

Thus, we have to find the largest delay largest combinational delay between any FF output and any FF input. We will be considering the following two paths

a. Path $Q_1$ to $D_2$:

$$t_{pdmax} = t_{p,AND,max} + t_{p,OR,max} = 20ps$$

b. Path $Q_2$ to $D_1$:

$$t_{pdmax} = t_{p,XOR,max} + t_{p,INV,max} = 25ps$$

The worst combination delay is 25 ps, with the given propagate delay from clock pin to Q $t_{pcq} = 10ps$ and the frequency $T_{clk} = 50ps$, we have

$$t_{setup} = T_{clk} - t_{pcq} - t_{pdmax} = 50 - 10 - 25 = \boxed{15ps}$$

Maximum setup time allowed for the flip-flops is 15ps.

## b

What is the maximum hold time of the flip-flops in order to avoid hold violation?

**Answer:** Alike the previous problem, we calculate the delay from the two path but now we calculate the fastest one to satisfy the following constraint

$$t_{hold} \leq t_{pcq} + t_{pdmin}$$

a. Path $Q_1$ to $D_2$:

$$t_{pdmax} = t_{p,AND,min} + t_{p,OR,min} = 10ps$$

b. Path $Q_2$ to $D_1$:

$$t_{pdmax} = t_{p,XOR,min} + t_{p,INV,min} = 12ps$$

The minimum combinational delay is the first path given by $10ps$. Hold is limited by the shortest path:

$$t_{holdmax} = min(t_{pdmin}) + t_{pcq} = 10 + 10 = \boxed{20ps}$$

The flip-flops must have a hold time $\leq 20ps$ to avoid hold violations in this circuit.

# Problem 3 - FSM Design

Design a Finite State Machine for a Button Press Synchronizer. This circuit should synchronize a button press to a clock signal, such that when the button is pressed, the output of this circuit is a signal that is '1' exactly for one clock cycle. Such a synchronized signal is useful to prevent a single button press that lasts multiple cycles from being interpreted as multiple button presses.

The circuit's input will be a signal $b_i$ and the output a signal $b_o$. When $b_i$ becomes 1, representing the button being pressed, the system should set $b_o$ to 1 for exactly one cycle. The system waits for $b_i$ to return to 0 again, and then waits for $b_i$ to become 1 again, which would represent the next pressing of the button.

A representative timing diagram of this behavior is shown below:

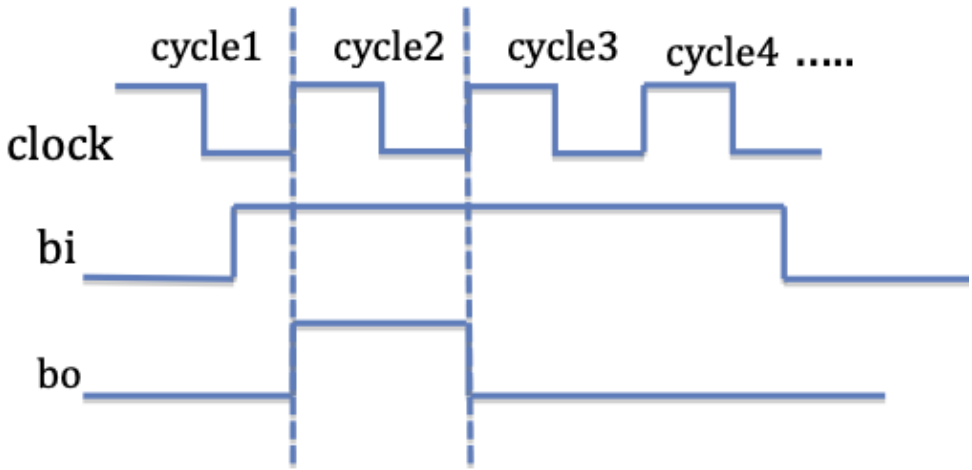


Figure 8: problem 3 behavior timing diagram

## a

Derive the FSM diagram. Refer to the lecture slides and the reading material (FSM Tutorial.pdf) posted on Canvas if you need pointers to how to convert a verbal description into a state diagram.

**Answer:**

From the description of the FSM and the timing behavior in Figure 8, the output $b_o$ depends only on the current state. In other words, the output does not change **immediately** based on the input

$b_i$, instead, $b_i$ only determines which state the FSM will transition to at the next rising clock edge. According to this behavior, we define the following states:

S0 : Wait For Press State

State Meaning: The button has not been pressed yey, waiting for $b_i = 1$. The current output $b_o = 0$.

State Behavior:If $b_i = 0$, we stay in the current state, and if $b_i = 1$, we go to state S1.

S1 : Pulse State

State Meaning: A new button press has been detected, and the FSM generates a one-cycle output pulse. Therefore, the current output is $b_o = 1$.

State Behavior: The next state will be S2 regardless of the input $b_i$.

S2 Wait For Release State

State Meaning: The button is still being held, and we've already produced the pulse; we must wait until it is released to return to S0.

State Behavior: If $b_i = 1$, stay in S2. If $b_i = 0$, transition back to S0 to wait for the next press.

The transitions described above correspond directly to the FSM diagram shown in Figure 9. As illustrated in the figure, S0 contains a self-loop for $b_i = 0$ and transitions to S1 when $b_i = 1$. State S1 produces the one-cycle pulse and then always transitions to S2. Finally, S2 remains active while the button is held ($b_i = 1$) and returns to S0 once the button is released ($b_i = 0$), completing the full button-press detection cycle.

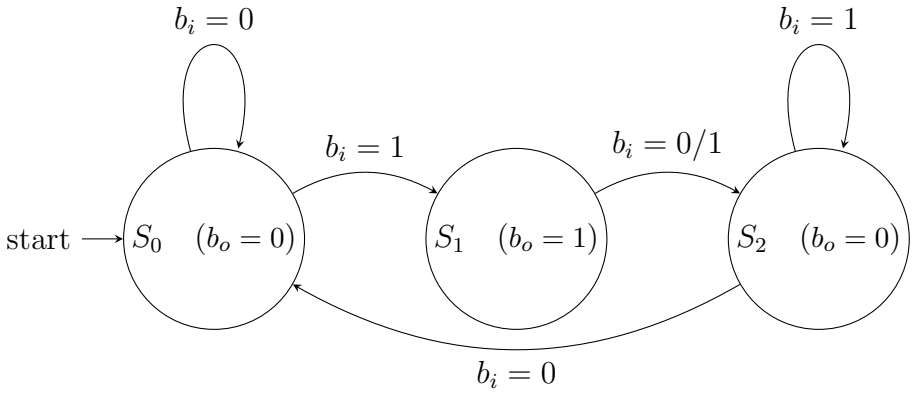


Figure 9: Button Press Synchronizer FSM (Moore machine)

## b

Derive the transition table and assign binary codes of your choice to states.

**Answer:**

We assign the following binary codes to the three states:

$$S_0 = 00, \quad S_1 = 01, \quad S_2 = 10.$$

Where the code 11 is unused. The table with the assigned binary code for states are shown in Table 1.

Table 1: State transition table for the button press synchronizer

| State (code) | $b_i$ | Next State | $b_o$ |
|---|---|---|---|
| $S_0$ (00) | 0 / 1 | $S_0$ (00) / $S_1$ (01) | 0 / 0 |
| $S_1$ (01) | 0 / 1 | $S_2$ (10) / $S_2$ (10) | 1 / 1 |
| $S_2$ (10) | 0 / 1 | $S_0$ (00) / $S_2$ (10) | 0 / 0 |

**c**

From this table derive the next state and output logic expression and draw the resulting circuit diagram of the sequential circuit.

**Answer:**

In the FSM described above, we use two state bits $Q_1$ and $Q_0$ to encode the three states.Based on the transition table in Table 1, we now derive the logic expressions for the output $b_o$, and the next-state bits $Q_0^+$, and $Q_1^+$. The derivation for each signal is summarized below.

$b_o$: From Table 1, the output $b_o$ is asserted only in state $S_1$. whose state encoding is $(Q_1, Q_0) = (0, 1)$. Therefore, the output depends solely on the current state bits and is given by the Boolean expression:

$$b_0 = \overline{Q_1} \cdot Q_0$$

$Q_0^+$ : The next-state bit $Q_0^+$ becomes 1 only when the FSM transitions from $S_0 \rightarrow S_1$, which occurs when the current state is $(Q_1, Q_0) = (0, 0)$ and the input $b_i = 1$. This yields the Boolean expression:

$$Q_0^+ = \overline{Q_1} \cdot \overline{Q_0} \cdot b_i$$

$Q_0^+$ : The next-state bit $Q_1^+$ becomes 1 in two situations:

- When transitioning from state $S_1 \rightarrow S_2$, which always occurs regardless of $b_i$
- When remaining in $S_2$ due to the input $b_i = 1$

Combining both cases from the transition table, the resulting logic expression is:

$$Q_1^+ = Q_0 + Q_1 \cdot b_i.$$

The resulting sequential circuit implementing the derived Boolean expressions is shown in Figure 10.

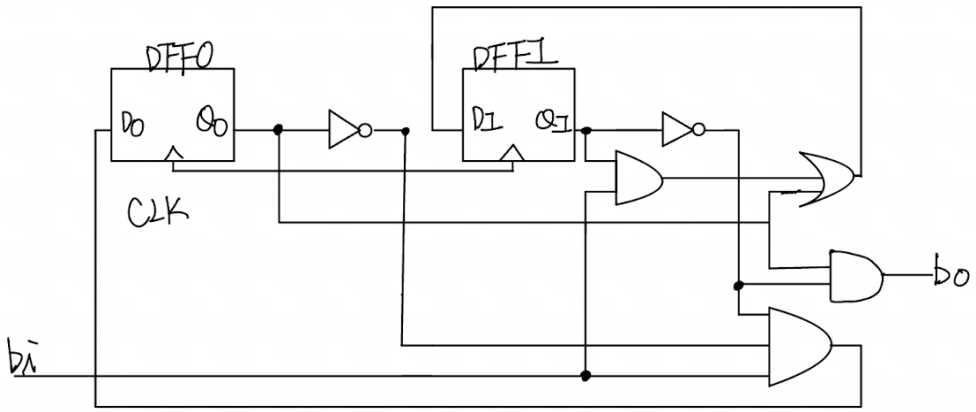


Figure 10: Circuit implementation of the button press synchronizer FSM.

## d

Implement the same FSM in Verilog using always statements and verify the behavior shown in the timing diagram above is matching your Verilog simulation. Submit all code and screenshot of simulation.

**Answer:**

The FSM was implemented in Verilog using a sequential always block for the state register and a combinational always block for the next-state and output logic. The testbench generates the clock, reset, and several button press patterns to verify the design behavior.

Figure 11a shows the RTL implementation of the FSM. Figure 11b shows the Verilog testbench used to stimulate the design. The simulated waveforms in Figure 12 confirm that the implementation matches the intended behavior: `b_o` is asserted for exactly one clock cycle after each rising edge of `b_i`, and no additional pulses are generated while `b_i` stays high.

Problem3 › button_sync.v

```
1  `timescale 1ns/1ps
2  module button_sync (b_o, clk, rst_n, b_i);
3      input   clk, rst_n, b_i;
4      output reg  b_o;
5
6      localparam S0_WAIT_PRESS   = 2'b00;
7      localparam S1_PULSE        = 2'b01;
8      localparam S2_WAIT_RELEASE = 2'b10;
9
10     reg [1:0] state, next_state;
11     always @(posedge clk or negedge rst_n) begin : state_reg
12         if (!rst_n)
13             state <= S0_WAIT_PRESS;
14         else
15             state <= next_state;
16     end
17     always @(*) begin : next_state_logic
18         next_state = state;
19         b_o        = 1'b0;
20         case (state)
21             S0_WAIT_PRESS: begin
22                 b_o = 1'b0;
23                 if (b_i == 1'b1)
24                     next_state = S1_PULSE;
25                 else
26                     next_state = S0_WAIT_PRESS;
27             end
28             S1_PULSE: begin
29                 b_o        = 1'b1;
30                 next_state = S2_WAIT_RELEASE;
31             end
32             S2_WAIT_RELEASE: begin
33                 b_o = 1'b0;
34                 if (b_i == 1'b0)
35                     next_state = S0_WAIT_PRESS;
36                 else
37                     next_state = S2_WAIT_RELEASE;
38             end
39             default: begin
40                 b_o        = 1'b0;
41                 next_state = S0_WAIT_PRESS;
42             end
43         endcase
44     end
45 endmodule
```

(a) Verilog RTL implementation of the button press synchronizer FSM.

Problem3 › button_sync_tb.v

```
1  `timescale 1ns/1ps
2  module button_sync_tb;
3
4      reg clk;
5      reg rst_n;
6      reg b_i;
7      wire b_o;
8
9      button_sync dut (
10         .clk  (clk),
11         .rst_n(rst_n),
12         .b_i  (b_i),
13         .b_o  (b_o)
14     );
15
16     initial begin
17         clk = 1'b0;
18         forever #5 clk = ~clk;   // 5ns high, 5ns low
19     end
20
21     initial begin
22         rst_n = 1'b0;
23         b_i   = 1'b0;
24         #20;
25         rst_n = 1'b1;
26         #7  b_i = 1'b1;
27         #30 b_i = 1'b0;
28         #20;
29         #3  b_i = 1'b1;
30         #50 b_i = 1'b0;
31
32         #15 b_i = 1'b1;
33         #10 b_i = 1'b0;
34
35         #30;
36         $finish;
37     end
38
39     integer cycle = 0;
40     always @(posedge clk) begin
41         cycle = cycle + 1;
42         $display("T=%0t ns, cycle=%0d, b_i=%b, b_o=%b",
43                  $time, cycle, b_i, b_o);
44     end
45
46 endmodule
```

(b) Testbench used to verify the button press synchronizer.

Figure 11: RTL and testbench code for Button Press Synchronizer

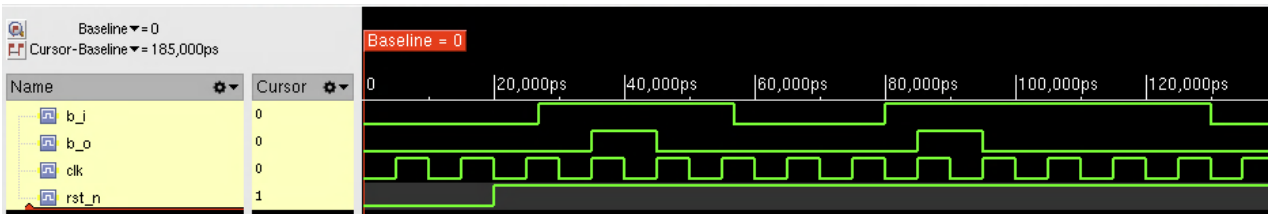


Figure 12: Simulation waveform showing a one-cycle pulse on $b_o$ for each rising edge of $b_i$.